

Assignment #2

Q1. Location / URL Launcher

In this assignment, it is your goal to attempt to reproduce the location box in your browser. You should have an edit box where the user can type in the URL or a web site. You will want to have a "Open URL" button to the right of the edit control. Clicking on this button should open a new window and load the URL.

Below is an example of the form...

www.medipol.edu.tr	open
---	-------------------------------------

Q2. Homemade Arithmetic Calculator

Create a simple arithmetic calculator that will do addition, subtraction, multiplication, and division. The user should be able to enter two numbers and then

Tables should be used so labels and controls align correctly.

Extra Checks:

All values must be positive. Provide necessary checks.

Remember to trap for the "divide by zero" error.

The form should resemble:

First Num	8
Second Num	9
Operation	Addition ▼
	Calculate
Result	17

Q3. Form Validation and DOM

Build a simple **user registration form** that **validates** user inputs in real-time and **displays** a success message once the form is correctly submitted.

Requirements

1. Create an HTML form with the following fields:
 - **Username** (required, minimum 4 characters)
 - **Email** (required, must be a valid email format)
 - **Password** (required, minimum 6 characters)
 - **Confirm Password** (must match Password)
 - **Submit Button**
2. **DOM Manipulation and Validation Rules:**
 - When the user starts typing, show instant feedback (e.g., "Username too short" if less than 4 characters).
 - When invalid input is detected, show a red border around the input box.
 - When input is valid, show a green border.
 - If passwords do not match, display an error message below the Confirm Password field.
 - Prevent form submission if any fields are invalid.
 - If all inputs are valid, show a **success message** (e.g., "Registration successful!") below the form without refreshing the page.

Starter HTML Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Registration Form</title>
  <style>
    .error { border: 2px solid red; }
    .success { border: 2px solid green; }
    .message { color: red; font-size: 0.9em; }
  </style>
</head>
<body>

  <h2>Register</h2>
  <form id="registrationForm">
    <label>Username:</label><br>
    <input type="text" id="username"><br>
    <span id="usernameMessage" class="message"></span><br>
```

```
<label>Email:</label><br>
<input type="email" id="email"><br>
<span id="emailMessage" class="message"></span><br>

<label>Password:</label><br>
<input type="password" id="password"><br>
<span id="passwordMessage" class="message"></span><br>

<label>Confirm Password:</label><br>
<input type="password" id="confirmPassword"><br>
<span id="confirmPasswordMessage" class="message"></span><br><br>

<button type="submit">Submit</button>
<button type="button" id="resetBtn">Reset</button>
</form>

<div id="result"></div>

<script src="rsapp.js"></script>
</body>
</html>
```

What you should implement in *rspp.js*

- Add event listeners for `input` and `submit`.
- Write validation functions for each field.
- Show dynamic feedback by manipulating the DOM (changing classes, adding/removing messages).
- Handle successful form submission without page refresh.
- Functions should be used whenever possible.

Hints:

1. Access the DOM elements:

- Use `document.getElementById()` or `querySelector()` to get each input field and message span.

2. Validation Functions:

- Create separate functions:

- `validateUsername()`
- `validateEmail()`
- `validatePassword()`
- `validateConfirmPassword()`

3. Add Event Listeners:

- Use `input` event for real-time validation.
- Use `submit` event to validate everything before submitting.

4. Manipulate Classes:

- If invalid: `input.classList.add('error')`
- If valid: `input.classList.remove('error');`
`input.classList.add('success')`

5. Display Error Messages:

- Set the `.innerText` of the corresponding `<span>` elements.

6. On Successful Validation:

- Prevent default form behavior (`event.preventDefault()`).
- Show a message like:

```
html
CopyEdit
<p style="color: green;">Registration Successful!</p>
```

7. Reset Button (Bonus):

- Clear all fields and remove classes/messages.